

Python Essentials I—Variables, Data Types, Operators & Input/Output

By the end of today you will understand what variables are, how Python stores different types of information, how to do calculations and comparisons, and how to take input from a user and display output back to them. You will write your first real Python program.

Quick recap from Day 1 — You installed Python, VS Code, Git, and GitHub. Today we write actual code for the first time. Open VS Code and create a new file called `day2_practice.py`. Every example in these notes goes in that file. Type it out — do not copy-paste. Typing forces your brain to process it.

SECTION 01 What is a Variable? How Python Stores Information

Before we look at any code, let us understand the concept of a variable in plain English. This is the single most important idea in all of programming.

The real-world definition of a variable

Think of your smartphone's contact list. When you save a contact, you give it a name — "Mum" — and behind that name you store a phone number: 9876543210. Whenever you want to call her, you search for "Mum" and the phone looks up the number automatically.

A variable is exactly the same thing. It is a name that points to a piece of information stored in the computer's memory. Instead of remembering the raw information yourself, you give it a label and refer to it by that label throughout your program.

In Python, creating a variable looks like this:

```
name = "Rahul"  
age  = 22  
city = "Bengaluru"
```

What is happening here:

- `name` is the label (the variable name)
- `=` is the assignment operator — it means "store this value"
- `"Rahul"` is the value being stored

After this, whenever Python sees the word name anywhere in your code, it goes to memory and gets back "Rahul". You never have to remember the actual value — just the label.

Think of a variable like a labelled box. You write "AGE" on the outside of the box and put the number 22 inside. Later, when you need the age, you just open the box labelled AGE. You can also replace what is inside the box at any time — that is why it is called a variable.

Rules for naming variables in Python

You can name a variable almost anything, but there are a few rules Python enforces and a few best practices professionals follow.

Rule	Good example	Bad example
Must start with a letter or underscore	student_name, _count	1name, 2age
Cannot have spaces — use underscore instead	first_name	first name
Cannot use Python keywords	my_list	list, if, for, True
Case-sensitive (Name and name are different)	student_name	StudentName (same var?)
Use lowercase with underscores (best practice)	monthly_salary	MonthlySalary

Python keywords are words the language has reserved for itself — like if, for, while, True, False, None. You cannot use these as variable names. If you type one in VS Code it will turn a different colour — that is your hint.

SECTION 02 Data Types — The 4 Types of Information Python Understands

Not all information is the same. The number 22 is different from the word "twenty-two". The answer True is different from both. Python needs to know what kind of information it is dealing with so it can handle it correctly.

A data type tells Python what kind of value is stored in a variable, and what operations are allowed on it. You cannot multiply two names together, but you can multiply two numbers. Python uses data types to enforce this logic.

Data Type 1 — Integer (int)

An integer is any whole number — positive, negative, or zero. No decimal point. No fractions.

int — Integer

Definition: A whole number with no decimal point. Can be positive, negative, or zero.

Example: 22, -5, 0, 1000, -999 are all integers.

Real life: your age, number of students in a class, number of items in a cart, year (2024), floor number in a building — all integers.

```
age          = 22
students     = 45
year         = 2024
temperature  = -5
items_in_cart = 0

print(age)          # Output: 22
print(type(age))    # Output: <class 'int'>
```

Output:

```
22
<class 'int'>
```

The `type()` function tells you what data type a variable is. You will use this constantly when debugging. If something is behaving unexpectedly, check its type first.

Data Type 2 — Float (float)

A float is a number with a decimal point. The name comes from "floating point" — the decimal point can float (move around). Use floats whenever you need precision beyond whole numbers.

float — Floating Point Number

Definition: A number that includes a decimal point. Used when you need precision.

Example: 3.14, 9.8, -2.5, 99.99, 0.001 are all floats.

Real life: price of a product (₹249.99), your GPA (8.7), temperature (36.6 degrees), distance in km (12.4), salary in lakhs (6.5) — all floats.

```
price        = 249.99
gpa          = 8.7
temperature  = 36.6
pi           = 3.14159

print(price)          # Output: 249.99
print(type(price))    # Output: <class 'float'>
```

Output:

```
249.99
<class 'float'>
```

Common confusion: 22 is an int. 22.0 is a float. They look almost the same but Python treats them differently. When in doubt, use a float for any real-world measurement or money value.

Data Type 3 — String (str)

A string is any sequence of characters — letters, numbers, spaces, symbols — wrapped inside quotes. Strings are how Python handles text. If it has quotes around it, it is a string, even if it looks like a number.

str — String

Definition: A sequence of characters enclosed in single quotes or double quotes. Used for any text.

Example: "Hello", 'Rahul', "Bengaluru", "9876543210", "AI is cool!" are all strings.

Real life: names, addresses, email addresses, phone numbers (stored as text, not for math), sentences, paragraphs, tweets, product descriptions — all strings.

```
name      = "Rahul Sharma"
city      = 'Bengaluru'
email     = "rahul@gmail.com"
phone     = "9876543210"      # String, not int — we won't do math on it
message   = "Welcome to Day 2!"

print(name)          # Output: Rahul Sharma
print(type(name))    # Output: <class 'str'>
```

Output:

```
Rahul Sharma
<class 'str'>
```

Single quotes and double quotes both work for strings in Python. Choose one and stay consistent. Double quotes are more common when your string contains an apostrophe: "I'm learning Python" works fine with double quotes.

Useful string tricks you need to know

Strings are the most common data type in real-world programming. Here are the essential operations:

```
# 1. Joining two strings (called concatenation)
first_name = "Rahul"
last_name  = "Sharma"
full_name  = first_name + " " + last_name
print(full_name)           # Output: Rahul Sharma

# 2. Finding the length of a string
print(len(full_name))      # Output: 12

# 3. Repeating a string
print("Ha" * 3)            # Output: HaHaHa

# 4. f-strings — the modern, clean way to mix variables into text
age = 22
msg = f"My name is {full_name} and I am {age} years old."
print(msg)
```

Output:

```
Rahul Sharma
12
HaHaHa
My name is Rahul Sharma and I am 22 years old.
```

f-strings (format strings) are the cleanest way to build sentences with variables in Python. Put an f before the opening quote, then use curly braces {} around any variable name inside the string. Python will replace it with the actual value automatically.

Data Type 4 — Boolean (bool)

A boolean is the simplest data type in Python — it can only ever be one of two values: True or False. That is it. Nothing else. Booleans are used to represent yes/no, on/off, correct/incorrect answers.

bool — Boolean

Definition: A data type that can only be True or False. Named after mathematician George Boole.

Example: `is_student = True, has_paid = False, is_logged_in = True`

Real life: is the user logged in? (True/False), has the payment been made? (True/False), is the door open? (True/False), did the test pass? (True/False) — all booleans.

```
is_student   = True
has_paid     = False
is_adult     = True
exam_passed  = False
```

```
print(is_student)          # Output: True
print(type(is_student))    # Output: <class 'bool'>

# Booleans come from comparisons
age = 20
print(age >= 18)           # Output: True   (20 is >= 18)
print(age == 30)          # Output: False  (20 is not 30)
```

Output:

```
True
<class 'bool'>
True
False
```

Important: True and False must start with a capital letter in Python. true and false (lowercase) will give you an error. This trips up almost every beginner.

All 4 data types — side by side

Type	Keyword	Example values	When to use it
Integer	int	22, 0, -5, 2024	Whole numbers, counts, years, IDs
Float	float	3.14, 9.99, -2.5	Prices, measurements, percentages
String	str	"Hello", '9876543210'	Any text, names, messages, phone numbers
Boolean	bool	True, False	Yes/No answers, on/off states, flags

SECTION 03 Operators — How Python Does Calculations and Comparisons

Operators are symbols that tell Python to do something with values. There are three main groups you need today: arithmetic operators (for math), comparison operators (for asking questions), and logical operators (for combining conditions).

Group 1 — Arithmetic Operators (math)

These work exactly like a calculator. You already know most of them from school.

Symbol	Name	Example	What it does
+	Addition	5 + 3	Adds two numbers together → 8
-	Subtraction	10 - 4	Subtracts right from left → 6

*	Multiplication	6 * 7	Multiplies two numbers → 42
/	Division	15 / 4	Divides and gives decimal result → 3.75
//	Floor Division	15 // 4	Divides and drops the decimal → 3
%	Modulo	15 % 4	Remainder after division → 3
**	Exponent	2 ** 8	2 raised to the power of 8 → 256

Let us see all of them in action with a real-life scenario: a shopping bill calculator.

```
price_per_item = 150
quantity       = 4
discount       = 50

total_before_discount = price_per_item * quantity
print(total_before_discount)      # Output: 600

total_after_discount = total_before_discount - discount
print(total_after_discount)       # Output: 550

gst_rate = 0.18
gst_amount = total_after_discount * gst_rate
print(gst_amount)                 # Output: 99.0

final_bill = total_after_discount + gst_amount
print(f"Final bill: Rs. {final_bill}") # Output: Final bill: Rs. 649.0
```

Output:

```
600
550
99.0
Final bill: Rs. 649.0
```

Understanding floor division and modulo — the two tricky ones

These two come up constantly in programming. Let us understand them with a simple example.

Floor division (//) — divides and throws away the decimal.

Example: You have 17 chocolates and want to share equally among 5 friends.

17 // 5 = 3 (each friend gets 3 whole chocolates)

Modulo (%) — gives you the leftover after division.

17 % 5 = 2 (2 chocolates are left over after equal sharing)

```
chocolates = 17
friends    = 5
```

```

each_gets  = chocolates // friends    # 3
leftover   = chocolates % friends     # 2

print(f"Each friend gets {each_gets} chocolates")
print(f"{leftover} chocolates are left over")

# Modulo is also used to check if a number is even or odd
number = 27
if number % 2 == 0:
    print("Even number")
else:
    print("Odd number")    # This will print

```

Output:

```

Each friend gets 3 chocolates
2 chocolates are left over
Odd number

```

Group 2 — Comparison Operators (asking yes/no questions)

Comparison operators compare two values and always return a Boolean answer — either True or False. They are the foundation of all decision-making in programming.

Symbol	Name	Example	Result and meaning
==	Equal to	5 == 5	True — 5 is equal to 5
!=	Not equal to	5 != 3	True — 5 is not equal to 3
>	Greater than	10 > 7	True — 10 is greater than 7
<	Less than	3 < 8	True — 3 is less than 8
>=	Greater than or equal	5 >= 5	True — 5 is equal to 5 so condition met
<=	Less than or equal	4 <= 9	True — 4 is less than 9 so condition met

Very common mistake: = is assignment (putting a value in a box). == is comparison (asking 'are these two things equal?'). Using = when you mean == is one of the most frequent bugs beginners write. Remember: one equals sign stores, two equals signs compares.

Practical example — checking eligibility for a loan:

```

age          = 25
monthly_income = 35000
credit_score  = 720

```

```

print(age >= 21)           # True  - old enough to apply
print(monthly_income >= 30000) # True  - income meets requirement
print(credit_score >= 750)   # False - credit score too low
print(monthly_income == 35000) # True  - exact match check
print(age != 18)           # True  - age is not 18

```

Output:

```

True
True
False
True
True

```

Group 3 — Logical Operators (combining conditions)

Sometimes one condition is not enough. You need to check two or more things at the same time. Logical operators let you combine multiple True/False conditions into one final answer.

Operator	What it does	Example
and	True only if BOTH conditions are True	age >= 18 and has_id == True
or	True if AT LEAST ONE condition is True	has_cash == True or has_card == True
not	Flips True to False and False to True	not is_banned

Real-world analogy for logical operators:

and — A bank gives you a loan if you earn enough AND your credit score is good. Both must be true.

or — You can enter a club if you have a ticket OR you are on the guest list. Either one is enough.

not — The system allows access if you are NOT banned. Flips the condition.

Practical example — checking if a user can register for a course:

```

age          = 19
has_degree   = True
is_blacklist = False

# Can they register? Must be 18+ AND have a degree
can_register = (age >= 18) and (has_degree == True)
print(can_register)           # True - both conditions met

# Are they eligible OR special case?
eligible = (age >= 18) or (has_degree == True)
print(eligible)              # True - at least one is True

```

```
# not operator - allow if NOT blacklisted
can_access = not is_blacklist
print(can_access)          # True - they are not blacklisted
```

Output:

```
True
True
True
```

SECTION 04 Input and Output — Talking to the User

Every real program needs two things: a way to receive information from the user (input) and a way to show results back (output). In Python, this is extremely simple.

Output with print()

You have already used print() on Day 1. Let us now understand it properly so you can use it professionally.

```
# Basic print - just display a value
print("Hello, World!")

# Print a variable
name = "Priya"
print(name)

# Print multiple things on one line
age = 23
print("Name:", name, "| Age:", age)

# The clean, professional way - f-strings
print(f"Student: {name}, Age: {age}")

# Print with a custom separator (default separator is a space)
print("Python", "is", "fun", sep="-")

# Print without moving to the next line
print("Loading", end="...")
print("Done")
```

Output:

```
Hello, World!
Priya
Name: Priya | Age: 23
```

```
Student: Priya, Age: 23
Python-is-fun
Loading...Done
```

Input with input()

The `input()` function pauses your program and waits for the user to type something and press Enter. Whatever they type is captured as a string and stored in a variable.

Important: `input()` always gives you a string, even if the user types a number. If you want to do math with it, you must convert it first using `int()` or `float()`.

```
# Ask the user for their name
name = input("What is your name? ")
print(f"Hello, {name}! Welcome to the program.")
```

Output:

```
What is your name? Rahul
Hello, Rahul! Welcome to the program.
```

Converting input from string to number

This is one of the most important things to understand about `input()`. Study this carefully.

```
# WRONG - this will crash if user types a number
# age = input("Enter your age: ")
# print(age + 1)  # ERROR: can't add int to string

# CORRECT - convert to int first
age = int(input("Enter your age: "))
print(f"In 10 years you will be {age + 10} years old")

# For decimal numbers, use float() instead
height = float(input("Enter your height in metres: "))
print(f"Your height is {height} m")
```

Output:

```
Enter your age: 22
In 10 years you will be 32 years old
Enter your height in metres: 1.75
Your height is 1.75 m
```

The three conversion functions to remember:

`int(x)` — converts `x` to a whole number. `int("22")` gives 22

`float(x)` — converts `x` to a decimal number. `float("3.14")` gives 3.14

`str(x)` — converts `x` to text.

`str(22)` gives `"22"`

Putting it all together — a complete mini-program

Let us combine everything from today into one small, complete program. This is exactly the kind of thing you should be able to build after Day 2.

```
# Student Grade Calculator
# This program takes a student's marks and calculates their result

print("==== Student Grade Calculator =====")
print("")

# Step 1: Take input from the user
student_name = input("Enter student name: ")
marks_str    = input("Enter marks out of 100: ")

# Step 2: Convert marks to a number so we can do math
marks = float(marks_str)

# Step 3: Calculate percentage
percentage = (marks / 100) * 100

# Step 4: Determine if passed or failed (pass mark is 40)
passed = marks >= 40

# Step 5: Display the results
print("")
print(f"Student : {student_name}")
print(f"Marks    : {marks} / 100")
print(f"Passed   : {passed}")
```

Output:

```
==== Student Grade Calculator =====

Enter student name: Rahul
Enter marks out of 100: 73

Student : Rahul
Marks    : 73.0 / 100
Passed   : True
```

Read through this program line by line. Every single line uses something you learned today: variables, data types, `input()`, output, arithmetic, and a comparison operator. This is how real programs are built — one concept at a time, layered together.

Getting error messages is completely normal. Every developer gets them every single day. The skill is not to avoid errors — it is to read them and understand what went wrong.

When Python shows you an error, it always tells you: (1) which line the problem is on, (2) what type of error it is, and (3) a short description. Read all three before panicking.

The 4 most common errors for beginners

Error 1 — `SyntaxError: invalid syntax`

This means Python could not understand what you wrote. Usually a missing colon, bracket, or quote.

```
# WRONG
print("Hello World"      # Missing closing bracket

# CORRECT
print("Hello World")
```

Error 2 — `NameError: name 'x' is not defined`

You used a variable that does not exist yet. Usually a spelling mistake.

```
# WRONG
print(stuudent_name)    # Misspelled variable name

# CORRECT
student_name = "Rahul"
print(student_name)
```

Error 3 — `TypeError: can only concatenate str to str, not int`

You tried to mix a string and a number together directly without converting.

```
# WRONG
age = 22
print("I am " + age + " years old")    # Can't join str and int

# CORRECT — Option A: convert int to str
print("I am " + str(age) + " years old")

# CORRECT — Option B: use f-string (cleaner)
print(f"I am {age} years old")
```

Error 4 — ValueError: invalid literal for int()

You tried to convert a value to int() but it was not a valid number.

```
# WRONG — user typed text when a number was expected
age = int("twenty two")    # Can't convert words to int

# CORRECT — user must type digits: 22
age = int("22")           # Works fine
```

Every time you get an error: (1) read the last line of the error message first, (2) find the line number it mentions, (3) look at that exact line in your code. 90% of the time the answer is right there. Only search the internet after reading the error message fully.

SECTION 06 Interview Prep — Questions to Answer After Today

Practice saying these answers out loud. Speaking through code questions is a separate skill from writing code. You need both.

Q — What is a variable? Can you give an example?

A variable is a named container that stores a value in the computer's memory. Instead of remembering raw values, we give them labels and refer to those labels in our code. For example: `student_name = "Rahul"` creates a variable called `student_name` that holds the text `Rahul`. Later in the code, whenever we use `student_name`, Python automatically replaces it with `Rahul`.

Q — What is the difference between int, float, str, and bool?

`int` is a whole number like 22 or -5. `float` is a decimal number like 3.14 or 99.99. `str` is text enclosed in quotes like "Hello" or "9876543210". `bool` is either `True` or `False`, used for yes/no conditions. The key difference is what operations you can perform: you can do math with `int` and `float`, but not with `str`. Booleans are the result of comparison operations.

Q — What is the difference between = and == in Python?

Single equals `=` is the assignment operator. It stores a value into a variable. For example, `age = 22` puts the number 22 into the variable called `age`. Double equals `==` is the comparison operator. It checks whether two values are equal and returns `True` or `False`. For example, `age == 22` returns `True` if `age` is currently 22. Using `=` when you mean `==` is one of the most common beginner bugs.

Q — Why does input() always return a string? How do you handle it?

The `input()` function has no way of knowing what the user will type before they type it. To stay safe, it always captures the text as a string. If we need to do math with it, we must explicitly convert it using `int()` for whole numbers or `float()` for decimal numbers. For example: `age = int(input("Enter age: "))` converts whatever the user types into an integer. If we forget this conversion and try to add a number to the input, Python gives a `TypeError`.

SECTION 07 Words to Know

Term	Definition
Variable	A named label that stores a value in memory. Like a labelled box.
Data type	The kind of value stored — int, float, str, or bool.
int	Integer. A whole number with no decimal point.
float	Floating point. A number with a decimal point.
str	String. Any text enclosed in quotes.
bool	Boolean. A value that is either True or False only.
Operator	A symbol that performs an operation: +, -, *, /, ==, >, and, or, not.
Assignment (=)	Stores a value into a variable. <code>age = 22</code> puts 22 into <code>age</code> .
Comparison (==)	Asks if two values are equal. Returns True or False.
Modulo (%)	Returns the remainder after division. <code>17 % 5 = 2</code> .
f-string	A string that lets you embed variable values using {curly braces}.
<code>input()</code>	A function that pauses the program and waits for the user to type something.
<code>int()</code> / <code>float()</code> / <code>str()</code>	Type conversion functions. Convert a value from one data type to another.
<code>TypeError</code>	Error when you try to do an operation on the wrong data type.
<code>NameError</code>	Error when you use a variable name that has not been defined yet.

SECTION HW Homework — Submit Before Day 3

All exercises must be pushed to your GitHub repo in a file called `day2_homework.py` before midnight. This counts as your daily pass/fail submission.

Exercise 1 — Personal Info Printer (Beginner)

Create variables for your own name, age, city, and whether you are a student (True/False). Print all four in one sentence using an f-string.

Expected output example: My name is Rahul, I am 22 years old, from Bengaluru, and student status: True

Exercise 2 — Simple Calculator (Beginner-Intermediate)

Write a program that asks the user to enter two numbers and then prints: the sum, the difference, the product, the result of dividing the first by the second (as a float), the floor division result, and the remainder.

Hint: remember to convert both inputs to float so math works correctly.

Exercise 3 — Age Eligibility Checker (Intermediate)

Ask the user to enter their age. Using comparison and logical operators, print: whether they are old enough to vote (18+), whether they are old enough to rent a car (21+), and whether they are in the age range for a student discount (15 to 25 inclusive).

Hint: for the range check, use the and operator to combine two comparisons.

Exercise 4 — Shopping Bill (Intermediate)

Ask the user for: product name, price per item, and quantity. Calculate and display the subtotal, GST (18%), and grand total using formatted output with rupee symbol.

Stretch Challenge — Tier A students

Build a BMI calculator. Ask for name, weight in kg, and height in metres. Calculate BMI (weight / height squared). Print the BMI value and a category: Underweight if below 18.5, Normal if 18.5 to 24.9, Overweight if 25 to 29.9, Obese if 30 and above. Use comparison operators for each category check.

Hint: height squared in Python is `height ** 2`